Informatik I
Dr. Henrik Brosenne
Paul Sarnighausen-Cahn

Wintersemester 2025/26
Georg-August-Universität Göttingen
Institut für Informatik

Übungsblatt 09

Studentische Lehrveranstaltungsevaluation

Die studentische Lehrveranstaltungsevaluation findet im Zeitraum

14.01.2026, 03:00 Uhr bis 08.02.2026, 23:59 Uhr

online in der Stud.IP-Veranstaltung **Informatik I (WiSe 2025/26)** statt.
Weitere Informationen zur zentralen Lehrveranstaltungsevaluation durch Studierende finden Sie unter:
`https://www.uni-goettingen.de/lehrveranstaltungsevaluation`

Allgemeine Information zum Bestehen der Prüfungsvorleistung.
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

Theoretische Übungen

Abgabe bis Di., 03.02., 14 Uhr

Allgemein

Die Lösung dieser Aufgaben werden über das Lernmodul VIPS abgegeben.

Aufgabe 1 – 20 Punkte

Rekursiver Münzwechsel (i1-u-dp0)

In Übungsblatt 08 wurde der Münzwechsel mit einem Greedy-Algorithmus betrachtet. In dieser Aufgabe soll das Problem exakt mithilfe eines rekursiven Ansatzes gelöst werden.

Gegeben sei ein Münzsystem mit positiven ganzzahligen Münzwerten

$$C = \{c_1, c_2, \ldots, c_k\}$$

sowie ein Betrag $B \in \mathbb{N}$. Jede Münze darf beliebig oft verwendet werden.

Wir betrachten die folgende Zielfunktion: Für jeden Betrag b soll die minimale Anzahl an Münzen bestimmt werden, mit der sich b mithilfe der Münzen aus C darstellen lässt.

1. Definieren Sie ein geeignetes Teilproblem $OPT(b)$ für alle $0 \leq b \leq B$, wobei $OPT(b)$ die minimale Anzahl an Münzen zur Darstellung von b beschreibt. Legen Sie dabei explizit fest, welchen Wert $OPT(b)$ für nicht darstellbare Beträge annimmt. (4 Punkte)

2. Geben Sie eine Rekursionsgleichung für $OPT(b)$ für $b > 0$ an. Begründen Sie kurz, warum Ihre Rekursionsgleichung alle möglichen Darstellungen von b korrekt berücksichtigt. Die Rekursion soll alle möglichen Münzwerte berücksichtigen, die zu b beitragen können. (6 Punkte)

3. Nennen Sie die notwendigen Basisfälle der Rekursion und begründen Sie diese kurz. (4 Punkte)

4. Erklären Sie kurz, warum dieser rekursive Ansatz unabhängig vom verwendeten Münzsystem immer eine optimale Lösung findet. Gehen Sie dabei insbesondere auf das zugrunde liegende Optimalitätsprinzip ein und begründen Sie, warum ein Greedy-Algorithmus im Allgemeinen scheitern kann. (6 Punkte)

Aufgabe 2 – 16 Punkte

Rekursionsbaum und Laufzeitanalyse (i1-u-rec2)

Gegeben sei die folgende rekursive Funktion:

```
def strange(n):
    if n <= 1:
        return 1
    return strange(n//2) + strange(n//2) + n
```

1. Zeichnen Sie den Rekursionsbaum für den Aufruf `strange(8)`. Beschriften Sie die Knoten mit den jeweiligen Argumentwerten. (4 Punkte)

2. Geben Sie die Tiefe des Rekursionsbaums in Abhängigkeit von n an und begründen Sie Ihre Antwort kurz. (4 Punkte)

3. Bestimmen Sie die asymptotische Laufzeit der Funktion in Tilde-Notation. Begründen Sie Ihre Aussage mithilfe des Rekursionsbaums. (4 Punkte)

4. Begründen Sie, warum Memoisation bei dieser Funktion keinen asymptotischen Laufzeitvorteil bringt. (4 Punkte)

Aufgabe 3 – 14 Punkte

Rekursion auf Strukturen: Balancierte Klammern (i1-u-rec3)

Eine Zeichenkette aus den Zeichen `(` und `)` heißt balanciert, wenn sie rekursiv wie folgt definiert ist:

- die leere Zeichenkette ist balanciert,
- ist S balanciert, so ist auch (S) balanciert,
- sind S und T balanciert, so ist auch ST balanciert.

1. Beschreiben Sie informell (z. B. in Worten oder Pseudocode) einen naiven, direkt aus der obigen Definition abgeleiteten rekursiven Ansatz für eine Funktion `balanced(s)`, die überprüft, ob eine Zeichenkette balanciert ist. Effizienzüberlegungen sind hierbei nicht relevant. (6 Punkte)
2. Zeichnen Sie einen ausschnittsweisen Rekursionsbaum für den Aufruf `balanced("(((()(()(())))")`. Zeigen Sie exemplarisch, wie der Aufruf in rekursive Teilaufrufe auf Teilstrings zerfällt. Es ist nicht erforderlich, alle rekursiven Aufrufe vollständig darzustellen. (4 Punkte)
3. Diskutieren Sie kurz die Worst-Case-Laufzeit dieses rekursiven Ansatzes in Abhängigkeit von der Länge der Zeichenkette. Begründen Sie insbesondere, warum die Laufzeit stark anwächst. (4 Punkte)

Praktische Übung

Abgabe der Prüfsumme bis Di., 03.02., 14 Uhr
Testat ab Di., 03.02., 18 Uhr

Hilfe zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

Aufgabe 1 – 25 Punkte

Münzwechsel mit Dynamic Programming (i1-p-dp1)

Gegeben sei ein Münzsystem `coins`, dargestellt als Liste positiver ganzer Zahlen. Jede Münze darf beliebig oft verwendet werden. Zusätzlich sei ein Betrag $B \in \mathbb{N}$ gegeben.

1. Implementieren Sie eine Bottom-Up-Dynamic-Programming-Lösung, die die minimale Anzahl an Münzen bestimmt, mit der der Betrag B gewechselt werden kann. (10 Punkte)
2. Falls der Betrag nicht darstellbar ist, soll die Funktion einen geeigneten Wert (z. B. `None`) zurückgeben. (5 Punkte)
3. Erweitern Sie Ihre Lösung so, dass zusätzlich eine konkrete optimale Münzzusammenstellung ausgegeben wird. (10 Punkte)

Aufgabe 2 – 25 Punkte

Maximale Summe ohne benachbarte Elemente (i1-p-dp2)

Gegeben sei eine Liste ganzer Zahlen `A` mit Länge n.

Gesucht ist die maximale Summe einer Teilfolge von `A`, bei der keine zwei gewählten Elemente in der Liste direkt nebeneinander liegen. Es ist erlaubt, kein Element zu wählen (Summe 0).

1. Implementieren Sie eine rekursive Funktion, die die maximale Summe für ein Präfix der Liste berechnet (z. B. für die ersten i Elemente). Gehen Sie dabei von einem direkten rekursiven Ansatz ohne Zwischenspeicherung aus. (8 Punkte)
2. Implementieren Sie eine Dynamic-Programming Lösung für das gesamte Problem. (12 Punkte)
3. Vergleichen Sie beide Ansätze anhand geeigneter Beispiele und erläutern Sie kurz den Effizienzunterschied. (5 Punkte)